

санкт-петербургский национальный
исследовательский университет итмо

дисциплина: бэк-энд разработка

ОТЧЕТ

домашняя работа №2
проектирование и технический дизайн api

ВЫПОЛНИЛ:

Иванов Виктор

K3340

проверил:

Добряков Д. И.

санкт-петербург

2026 г.

задача

на основе разработанной в лабораторной работе №1 схемы базы данных необходимо:

1. описать функциональные и нефункциональные требования к api
2. выбрать архитектурный подход: rest api или backend for frontend (bff)
3. спроектировать openapi-спецификацию всех эндпоинтов
4. определить форматы запросов и ответов для каждого эндпоинта
5. описать возможные ошибки и коды возврата

ход работы

1. функциональные требования

fr-1: управление пользователями

- регистрация с указанием email, пароля, имени и телефона
- аутентификация по email и паролю с выдачей jwt токена
- просмотр и редактирование профиля

fr-2: поиск ресторанов

- поиск по названию, типу кухни, ценовому диапазону
- просмотр детальной информации: меню, фотографии, отзывы
- пагинация результатов поиска

fr-3: бронирование

- создание бронирования с указанием даты, времени, столика
- проверка доступности столиков
- просмотр истории бронирований
- отмена бронирования

fr-4: отзывы

- создание отзыва с рейтингом (1-5) и комментарием
- просмотр отзывов о ресторане
- автоматический пересчет рейтинга ресторана

2. нефункциональные требования

nfr-1: производительность

- время отклика api < 200ms для 95% запросов
- поддержка минимум 1000 запросов в секунду

nfr-2: безопасность

- jwt аутентификация для защищенных эндпоинтов
- хеширование паролей (bcrypt)

- rate limiting: 100 запросов/минуту на ip
- передача данных по https

nfr-3: надежность

- доступность 99.9%
- структурированное логирование всех запросов
- понятные сообщения об ошибках

nfr-4: совместимость

- openapi 3.0 спецификация
- json формат для всех запросов и ответов
- версионирование api через url (/api/v1/)

3. выбор архитектурного подхода

для проекта был выбран **rest api** подход по следующим причинам:

- **универсальность:** rest api может использоваться любыми клиентами (web, mobile, desktop)
- **стандартизация:** использование http методов (get, post, put, delete) делает api интуитивным
- **stateless:** каждый запрос независим, что упрощает масштабирование
- **кеширование:** rest поддерживает http кеширование

backend for frontend (bff) не подходит, так как:

- в проекте один основной клиент (веб-приложение)
- нет необходимости в композиции данных из множества микросервисов
- монолитная архитектура с прямой связью ui и бизнес-логики

4. основные принципы rest api

http методы:

- get — получение данных
- post — создание ресурса
- put/patch — обновление ресурса

- delete — удаление ресурса

структура url:

```
/api/v1/{resource}  
/api/v1/{resource}/{id}  
/api/v1/{resource}/{id}/{sub-resource}
```

КОДЫ ОТВЕТОВ:

- 200 ok — успешный запрос
- 201 created — ресурс создан
- 400 bad request — ошибка валидации
- 401 unauthorized — требуется аутентификация
- 404 not found — ресурс не найден
- 409 conflict — конфликт (например, столик занят)
- 500 internal server error — ошибка сервера

5. ОПИСАНИЕ ОСНОВНЫХ ЭНДПОИНТОВ

5.1. аутентификация

post /api/v1/auth/register

регистрация нового пользователя

запрос:

```
1 {  
2   "email": "user@example.com",  
3   "password": "securepass123",  
4   "first_name": "иван",  
5   "last_name": "петров",  
6   "phone": "+79161234567"  
7 }
```

ответ (201 created):

```
1 {  
2   "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
3   "token_type": "bearer",  
4   "expires_in": 86400,  
5   "user": {  
6     "id": "123e4567-e89b-12d3-a456-426614174000",  
7     "email": "user@example.com",
```

```

8     "first_name": "иван",
9     "last_name": "петров"
10 }
11 }

```

ошибки:

- 409 conflict — email уже существует
- 422 unprocessable entity — ошибка валидации

post /api/v1/auth/login

вход в систему

запрос:

```

1 {
2     "email": "user@example.com",
3     "password": "securepass123"
4 }

```

ответ (200 ok): аналогичен register

ошибки:

- 401 unauthorized — неверные credentials

5.2. рестораны

get /api/v1/restaurants

поиск ресторанов с фильтрацией

параметры запроса:

- search — поиск по названию
- cuisine_type — тип кухни
- price_range — ценовой диапазон (1-4)
- min_rating — минимальный рейтинг
- page — номер страницы (по умолчанию 1)
- page_size — размер страницы (по умолчанию 20)

пример: /api/v1/restaurants?cuisine_type=итальянская&min_rating=4

ответ (200 ok):

```

1 {
2     "items": [
3         {

```

```

4      "id": "123e4567-e89b-12d3-a456-426614174000",
5      "name": "ресторан пушкин",
6      "cuisine_type": "русская",
7      "price_range": 3,
8      "rating": 4.7,
9      "address": "москва, тверской бульвар, 26а"
10    }
11  ],
12  "pagination": {
13    "page": 1,
14    "page_size": 20,
15    "total_items": 156,
16    "total_pages": 8
17  }
18 }

```

get /api/v1/restaurants/{id}

получение детальной информации о ресторане

ответ (200 ok):

```

1 {
2   "id": "123e4567-e89b-12d3-a456-426614174000",
3   "name": "ресторан пушкин",
4   "description": "изысканная русская кухня",
5   "cuisine_type": "русская",
6   "price_range": 3,
7   "rating": 4.7,
8   "reviews_count": 342,
9   "photos_count": 28
10 }

```

ошибки:

- 404 not found — ресторан не найден

get /api/v1/restaurants/{id}/availability

проверка доступности столиков

параметры запроса:

- date — дата (yyyy-mm-dd)
- time — время (hh:mm)
- guests — количество гостей

пример: /api/v1/restaurants/{id}/availability?date=2025-04-15&time=19:00&guests=2

ответ (200 ok):

```

1 {
2   "available": true,
3   "tables": [
4     {
5       "id": "uuid",
6       "table_number": "5",
7       "capacity": 4
8     },
9     {
10      "id": "uuid",
11      "table_number": "7",
12      "capacity": 6
13    }
14  ]
15 }

```

5.3. бронирование

post /api/v1/bookings

создание бронирования (требуется аутентификация)

заголовок: authorization: bearer <token>

запрос:

```

1 {
2   "restaurant_id": "123e4567-e89b-12d3-a456-426614174000",
3   "table_id": "123e4567-e89b-12d3-a456-426614174001",
4   "booking_date": "2025-04-15",
5   "booking_time": "19:00",
6   "guests_count": 4
7 }

```

ответ (201 created):

```

1 {
2   "id": "uuid",
3   "user_id": "uuid",
4   "restaurant_id": "uuid",
5   "table_id": "uuid",
6   "booking_date": "2025-04-15",
7   "booking_time": "19:00",
8   "guests_count": 4,
9   "status": "pending",
10  "created_at": "2025-04-01T12:00:00z"
11 }

```


ошибки:

- 401 unauthorized — нет токена
- 404 not found — ресторан или столик не найден
- 409 conflict — столик недоступен на указанное время
- 422 unprocessable entity — ошибка валидации

get /api/v1/bookings

список бронирований пользователя (требуется аутентификация)

параметры запроса:

- status — фильтр по статусу (pending, confirmed, completed, cancelled)

ответ (200 ok):

```
1 {
2   "items": [
3     {
4       "id": "uuid",
5       "restaurant": {
6         "id": "uuid",
7         "name": "ресторан пушкин"
8       },
9       "table": {
10        "table_number": "5",
11        "capacity": 4
12      },
13       "booking_date": "2025-04-15",
14       "booking_time": "19:00",
15       "status": "confirmed"
16     }
17   ],
18   "pagination": {...}
19 }
```

delete /api/v1/bookings/{id}

отмена бронирования (требуется аутентификация)

ответ (204 no content) — бронирование отменено

ошибки:

- 401 unauthorized — нет токена
- 403 forbidden — нет доступа к этому бронированию
- 404 not found — бронирование не найдено
- 409 conflict — бронирование нельзя отменить (уже завершено)

5.4. отзывы

post /api/v1/restaurants/{id}/reviews

создание отзыва (требуется аутентификация)

запрос:

```
1 {
2   "rating": 5,
3   "comment": "отличный ресторан! прекрасная кухня."
4 }
```

ответ (201 created):

```
1 {
2   "id": "uuid",
3   "user": {
4     "first_name": "иван",
5     "last_name": "петров"
6   },
7   "rating": 5,
8   "comment": "отличный ресторан!",
9   "created_at": "2025-04-02t15:30:00z"
10 }
```

ошибки:

- 409 conflict — пользователь уже оставил отзыв
- 422 unprocessable entity — некорректный рейтинг или комментарий

get /api/v1/restaurants/{id}/reviews

получение отзывов о ресторане

параметры запроса:

- page, page_size — пагинация
- min_rating — фильтр по минимальному рейтингу

ответ (200 ok): список отзывов с пагинацией

6. формат ошибок

все ошибки возвращаются в едином формате:

```
1 {
2   "error": {
3     "code": "validation_error",
4     "message": "ошибка валидации входных данных",
5     "details": {
```

```

6     "email": ["некорректный формат email"],
7     "password": ["пароль должен содержать минимум 8 символов"]
8   }
9 }
10 }

```

типы ошибок:

- `validation_error` — ошибка валидации
- `unauthorized` — требуется аутентификация
- `forbidden` — нет прав доступа
- `not_found` — ресурс не найден
- `conflict` — конфликт состояния
- `internal_server_error` — ошибка сервера

7. openapi спецификация

полная openapi 3.0 спецификация всех эндпоинтов представлена в файле `openapi.yaml` в репозитории проекта. спецификация включает:

- описание всех 15 эндпоинтов api
- схемы данных для запросов и ответов
- описание параметров и заголовков
- примеры запросов и ответов
- описание всех возможных ошибок
- jwt аутентификацию через bearer token

спецификация может быть использована для:

- автоматической генерации документации (swagger ui)
- генерации клиентского кода
- валидации запросов и ответов
- тестирования api

ВЫВОД

в ходе выполнения лабораторной работы были:

1. определены требования к api:

- 4 группы функциональных требований (пользователи, рестораны, бронирование, отзывы)
- 4 группы нефункциональных требований (производительность, безопасность, надежность, совместимость)

2. выбран архитектурный подход:

- rest api как оптимальное решение для универсального доступа
- обоснован отказ от bff в пользу rest

3. спроектированы эндпоинты:

- 15 rest эндпоинтов, покрывающих весь функционал
- использование стандартных http методов (get, post, put, delete)
- версионирование через url (/api/v1/)

4. описаны форматы данных:

- json формат для всех запросов и ответов
- единый формат ошибок с кодами и описаниями
- jwt аутентификация для защищенных эндпоинтов

5. создана openapi 3.0 спецификация:

- полное описание всех эндпоинтов
- схемы данных и примеры запросов/ответов
- документация возможных ошибок

результат работы:

разработанное api обеспечивает:

- полную поддержку функциональных требований приложения
- безопасность через jwt аутентификацию
- масштабируемость за счет stateless архитектуры
- удобство интеграции благодаря rest принципам
- возможность автоматической генерации документации

следующие этапы разработки:

1. реализация api на выбранном backend фреймворке
2. настройка базы данных и orm
3. написание unit и integration тестов
4. настройка ci/cd pipeline
5. деплой на production окружение

полученная openapi спецификация будет использована как контракт между frontend и backend командами и как основа для автоматической генерации клиентского кода.